Université de Toulouse

Institut de Recherche en Informatique de Toulouse
CNRS - INP - UT3 - UT1 - UT2J

UPIPO : Unecessary Page In Page Out

io_uring based approach for double paging problem

Kilian Kemgne

February 16, 2026

UT / Toulouse INP/ IRIT / SEPIA
Supervisor: Prof. Daniel Hagimont
Co-supervisor: Prof. Alain Tchana

University of Toulouse

Contents

Swap process

Explanation

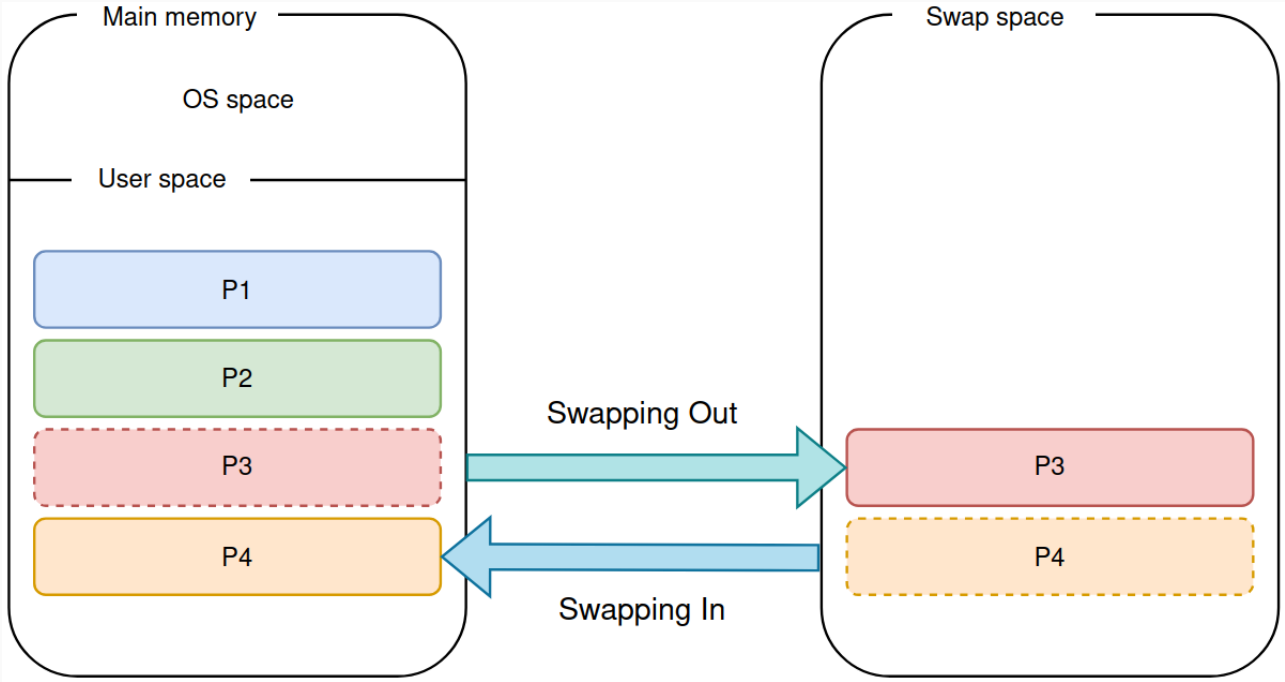


Figure 1: Swap In / Out process

Double Paging problem

Explanation

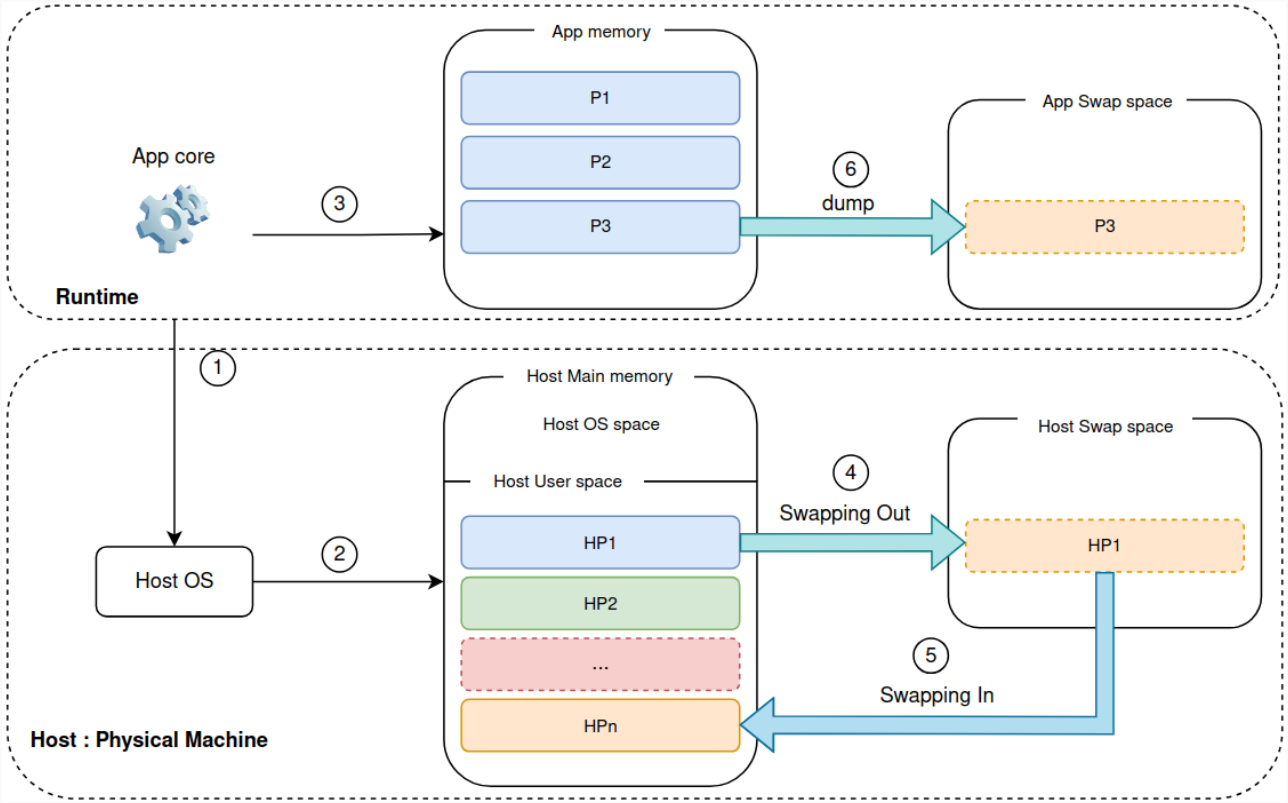
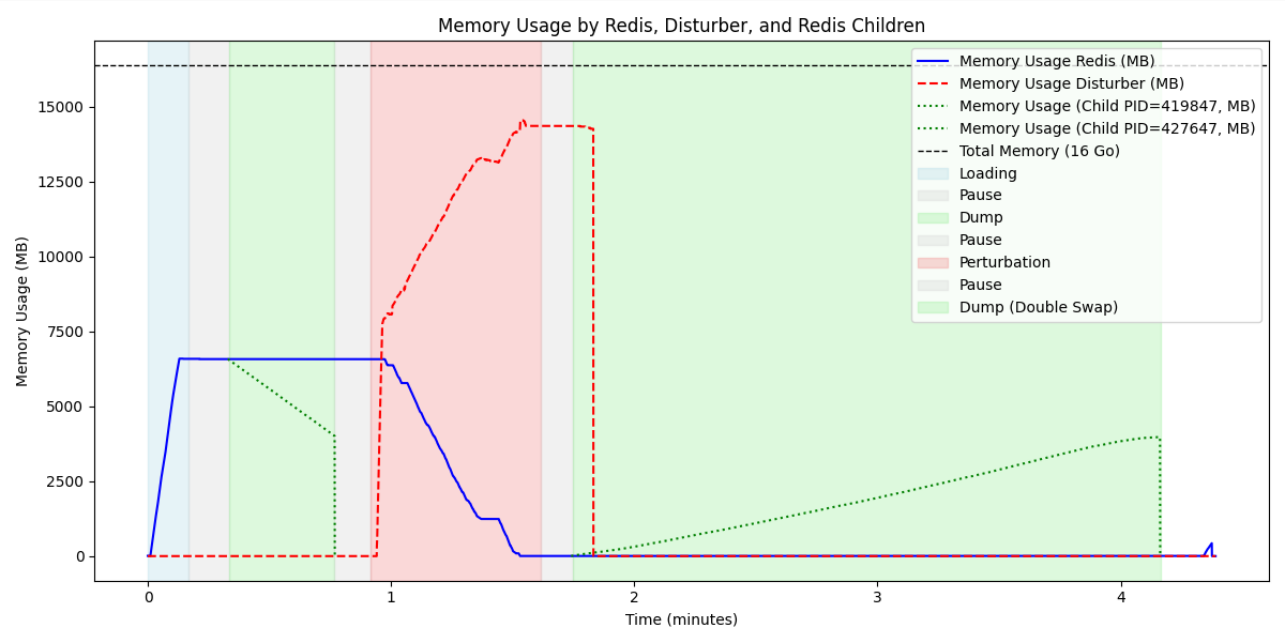


Figure 2: Double paging problem

Double Paging impact in certain runtimes

Redis : An in memory database



The dump with perturbation takes **5.58 times longer** *than the dump without perturbation*

Figure 3: Redis Memory Usage

Double Paging impact in certain runtimes

Redis : An in memory database

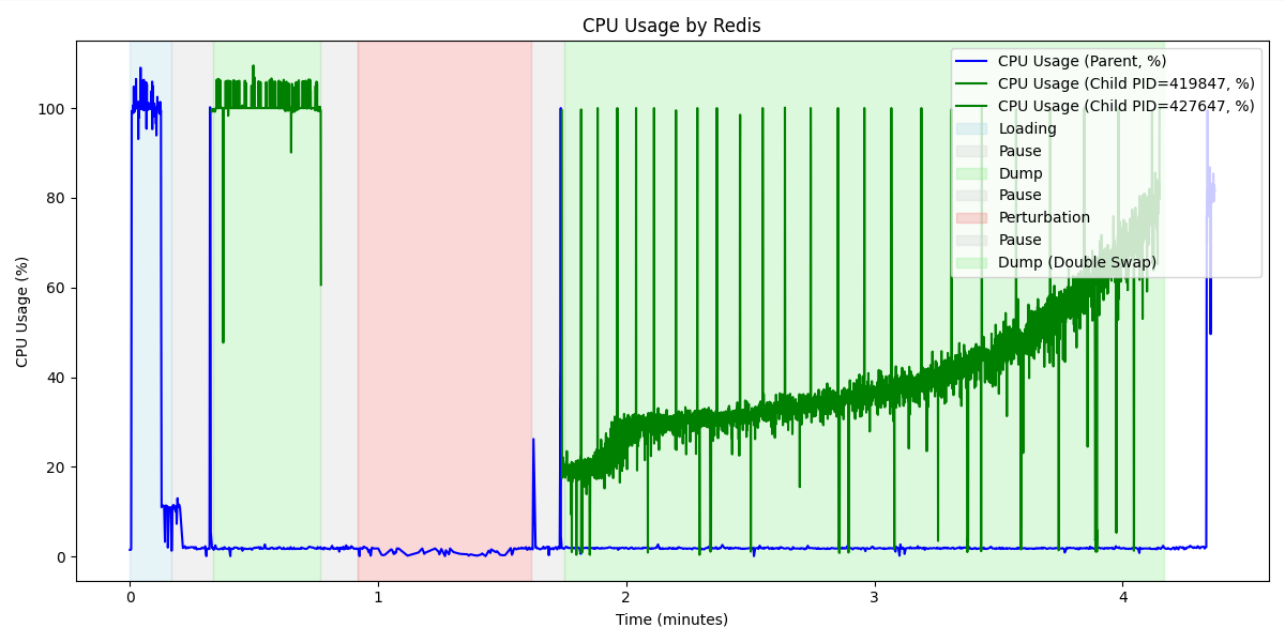
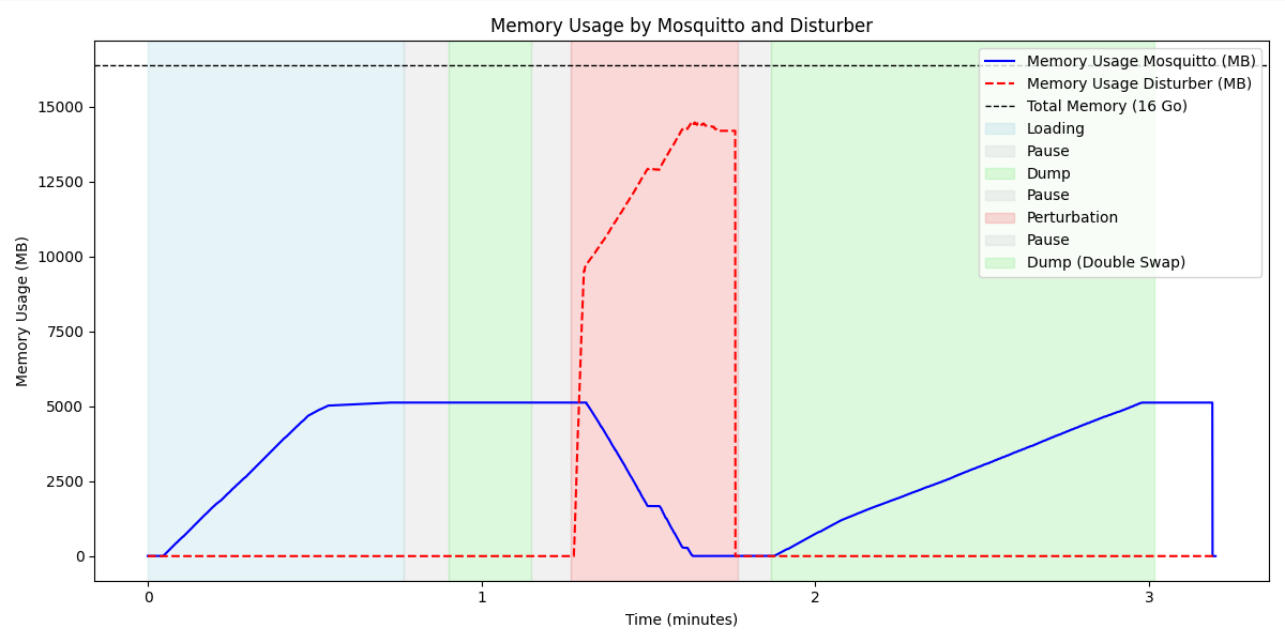


Figure 4: Redis CPU Usage

Double Paging impact in certain runtimes

Mosquitto :
Message broker based on MQTT protocol



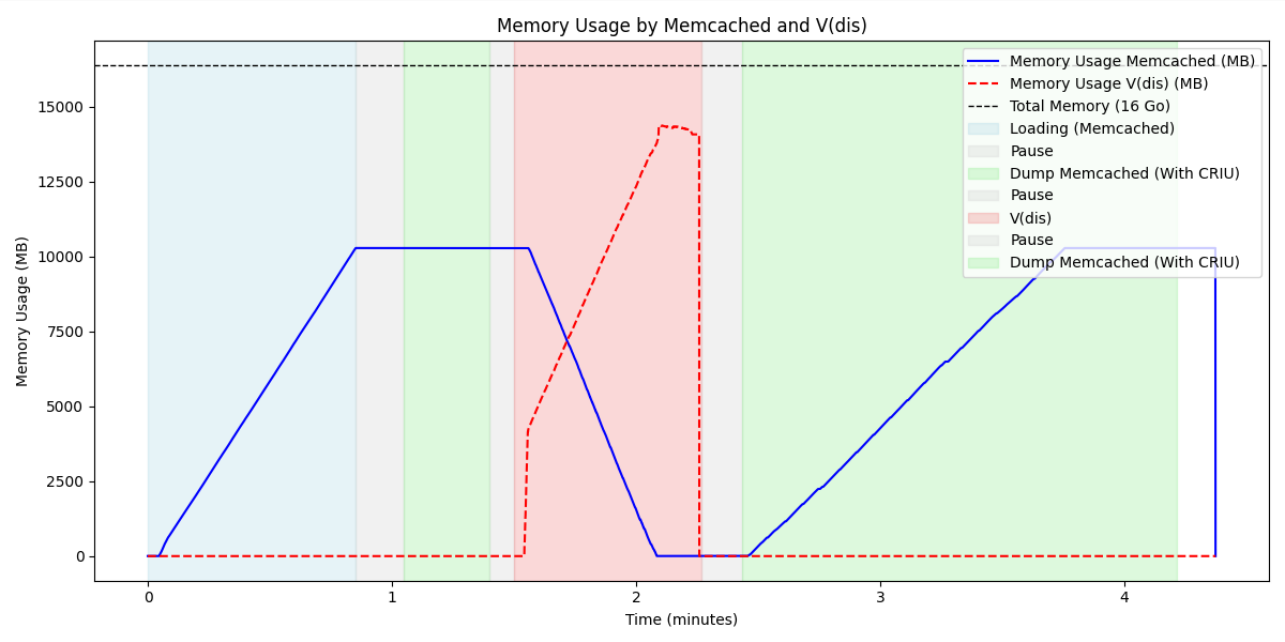
The dump with perturbation takes **4.6 times longer** *than the dump without perturbation*

Figure 5: Mosquitto Memory Usage

Université de Toulouse

Double Paging impact in certain runtimes

CRIU : Checkpoint / Restore in Userspace



The dump with perturbation takes **5.1 times longer** *than the dump without perturbation*

Figure 6: CRIU Memory Usage

Université de Toulouse

Challenges :

1. Semantic gap between paging and dumping processes (format, storage location, page tranversal)
2. Not All pages are affected by UPIPO
3. Multiple use cases to consider
4. Usability : Ensuring ease of integration with the application and OS
5. Security : avoiding vulnerabilities in the OS
6. Efficiency : Maintaining the same performance levels as traditional approaches
7. Correctness : Ensuring consistency of the application

- Communication mechanism
- Communication process
- User space API
- Kernel space API

Upipo Design

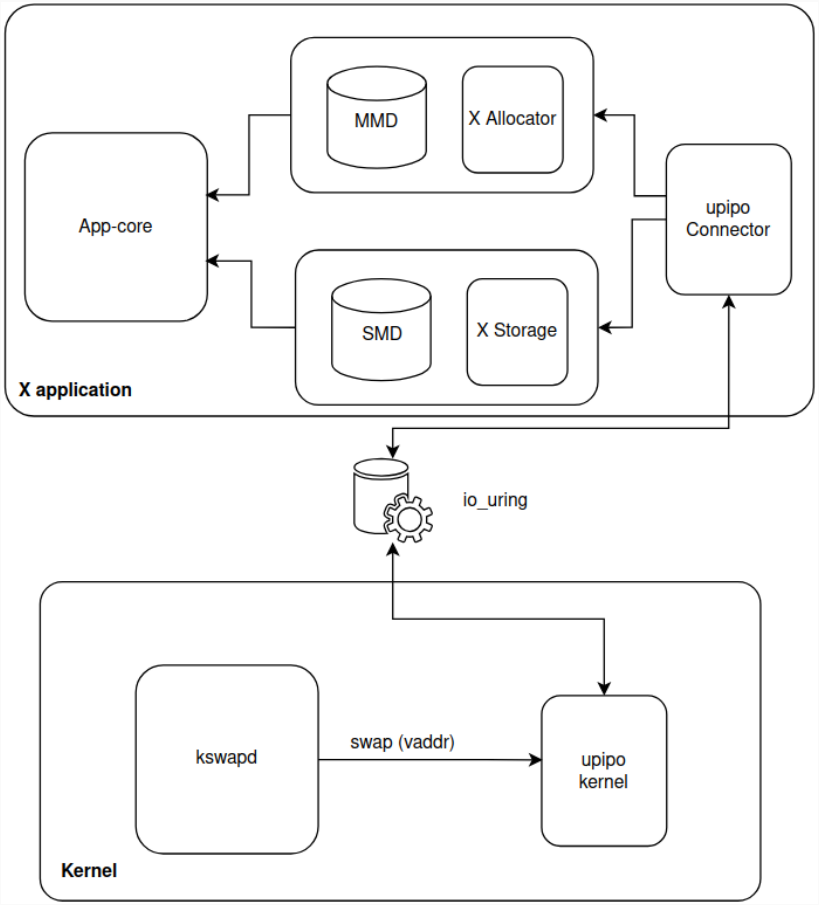


Figure 7: upipo global design

- Communication mechanism (6)
- Communication process
- User space API
- Kernel space API

Upipo Design

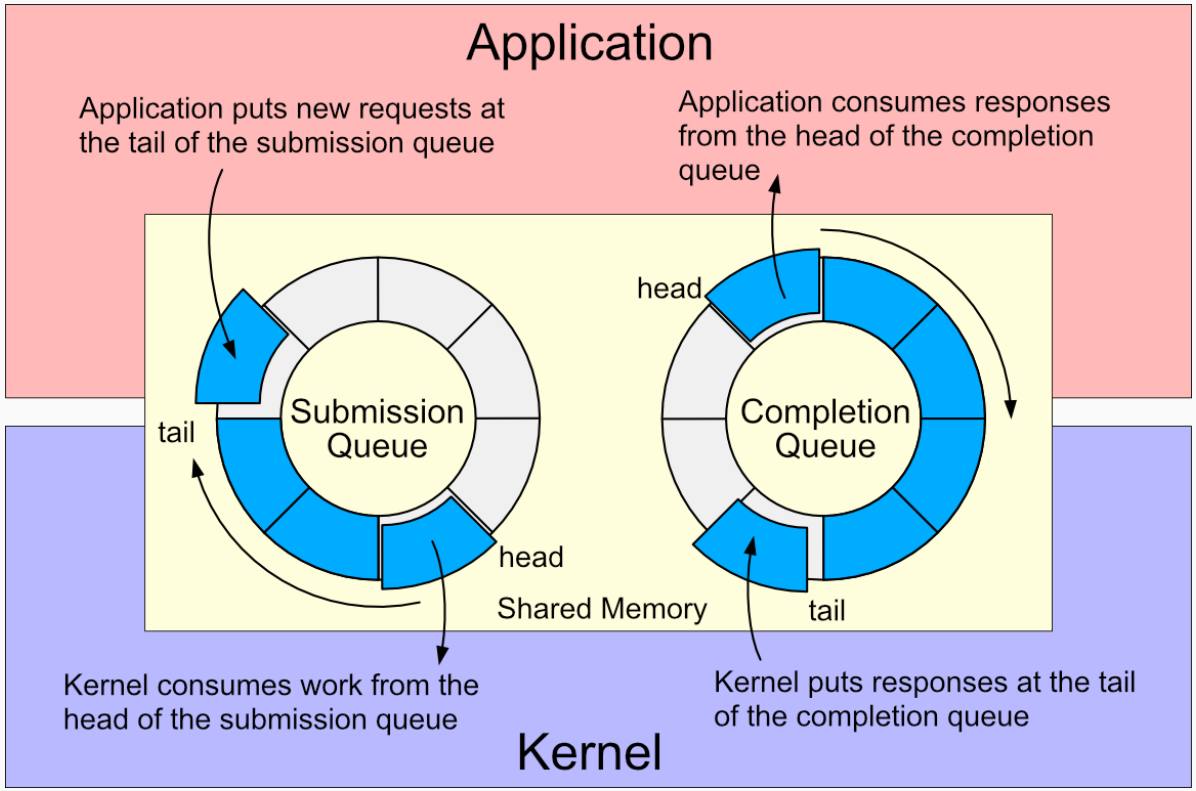


Figure 8: io_uring design

- Communication mechanism
- Communication process
- User space API
- Kernel space API

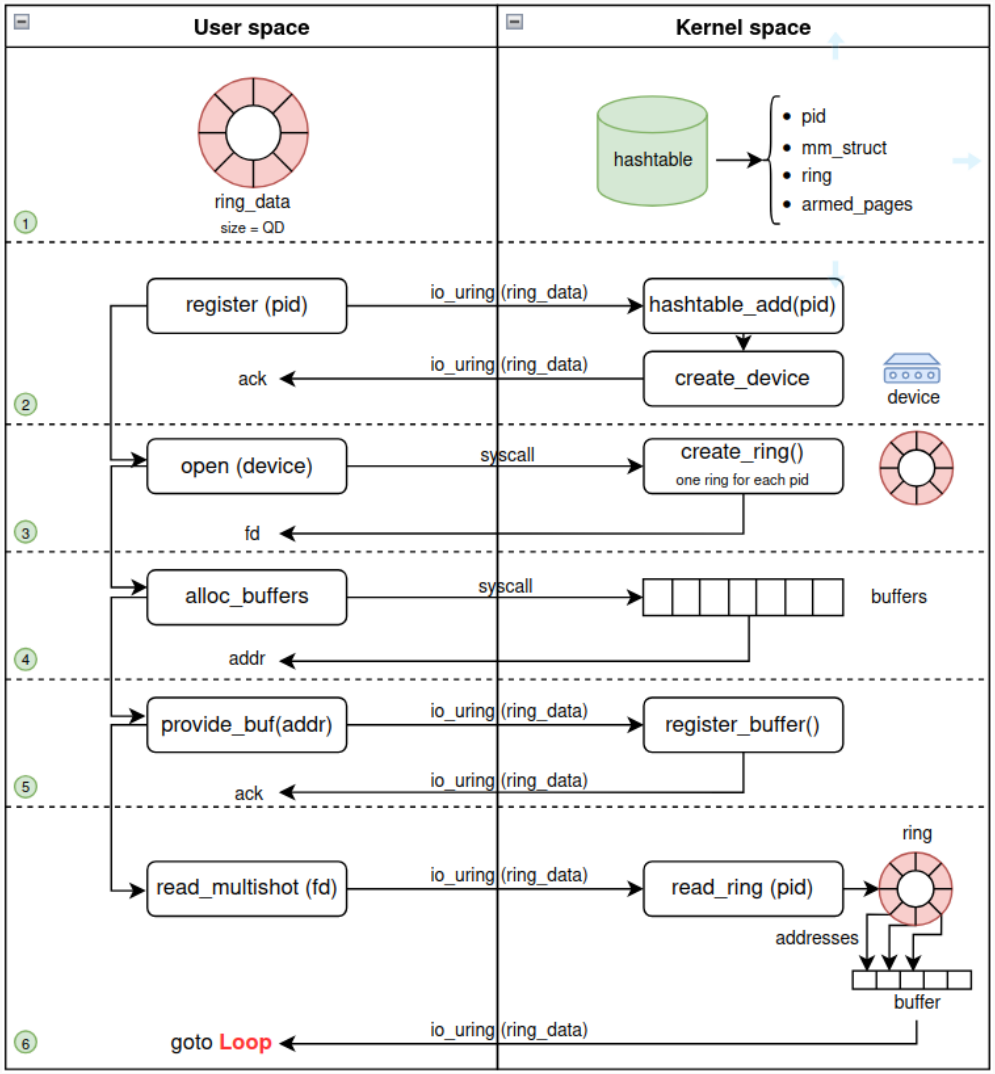


Figure 9: io_uring communication process

- Communication mechanism
- Communication process
- User space API
- Kernel space API

Upipo Design

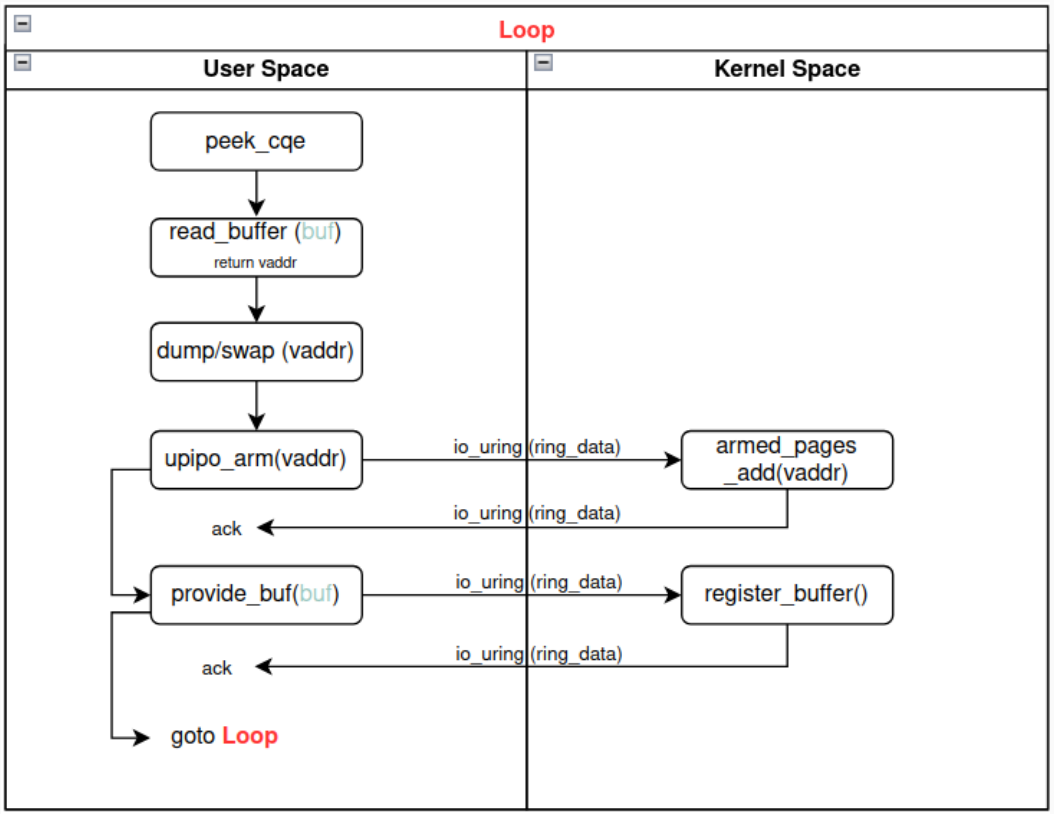


Figure 10: io_uring communication process

- Communication mechanism
- Communication process
- User space API (1,2,3,7)
  - upipo_connector
  - upipo_alloc
- Kernel space API

Upipo Design

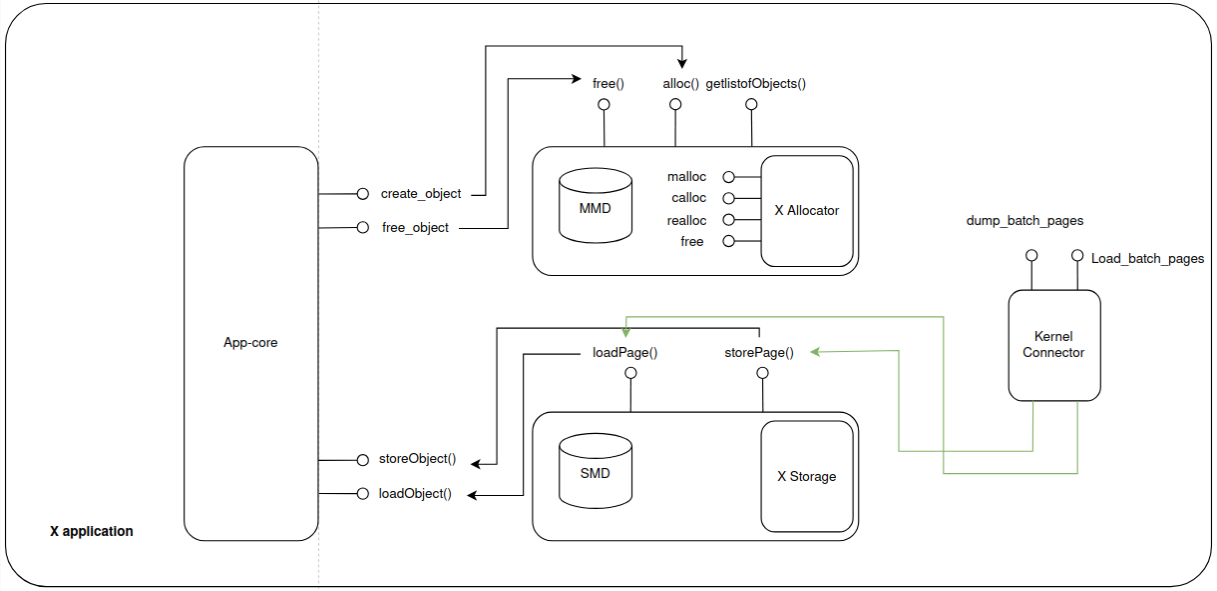


Figure 11: upipo_space API design

Upipo Design

- Communication mechanism
- Communication process
- User space API
  - upipo_connector
  - upipo_alloc
- Kernel space API

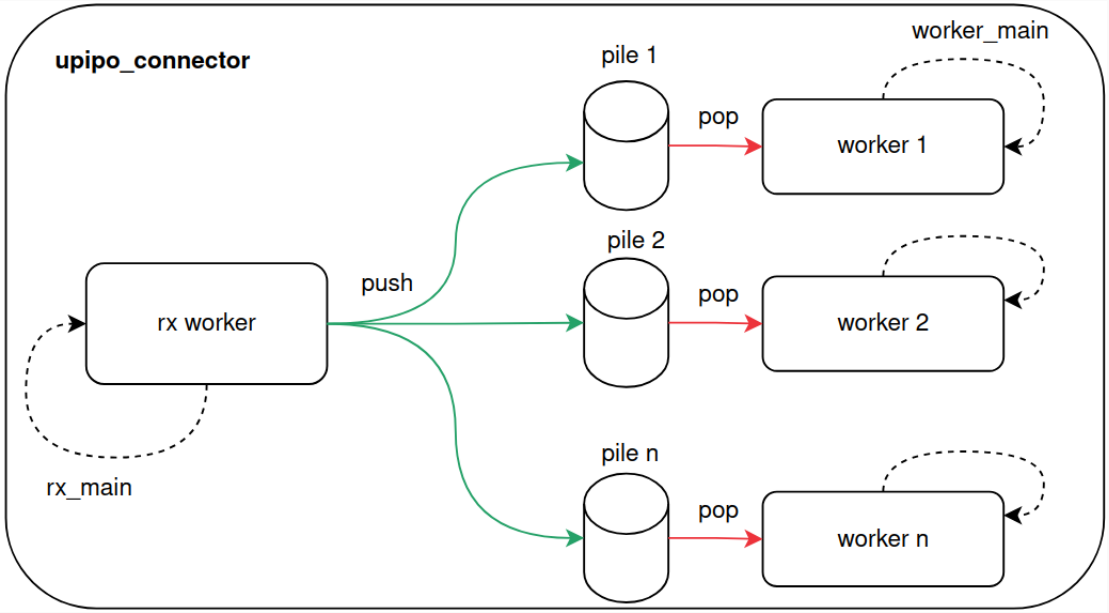


Figure 12: upipo_connector API

- Communication mechanism
- Communication process
- User space API
  - upipo_connector
  - upipo_alloc
- Kernel space API

Upipo Design

1. **struct upipo_connector_ctx**
   - prep_file (file)
   - dump_function (file, virt_addr)
   - load_function (file, virt_addr, buf)
2. **void * upipo_start_async(u_ctx)**

Université de Toulouse

- Communication mechanism
- Communication process
- User space API
  - upipo_connector
  - upipo_alloc
- Kernel space API

Upipo Design

- **alloc()** : application will it use to allocate persistent data memory
- **free()** : free pages
- **SMD** : storage MetaData
- **MMD** : memory MetaData

- Communication mechanism
- Communication process
- User space API
- Kernel space API

Upipo Design

1. **io_uring**
   - io_uring_prep_upipo_register / upipo_unregister
   - io_uring_prep_upipo_arm / upipo_disarm
   - io_uring_prep_upipo_swap_in
2. **upipo**
   - upipo_divert_folio for kswapd
   - devices_operations for users

On going work

Université de Toulouse

Institut de Recherche en Informatique de Toulouse
CNRS - INP - UT3 - UT1 - UT2J

Thank you for your attention

Kilian KEMGNE

kilian-lenaic.kemgne-tape@irit.fr